

EGCI341: Software Engineering (2026 Agile & DevOps Edition)

Project Goal: Gain direct experience in analysis, planning, and design for a software system where you play both the Customer (Product Owner) and the Software Engineer.

Max. pages: 20 pages

1. Project Report Guidance

The following sections must be included in your final report:

1.1 Problem Definition & AI-Assisted Ideation (5 Marks)

- **What to do:** Identify a real-world problem. Use AI tools (like Claude or ChatGPT) to brainstorm unique features or refine your problem statement.
- **Output:** Clear objectives and a list of User Stories (e.g., "As a [user], I want to [action] so that [value]").

1.2 Agile Feasibility & Project Roadmap (10 Marks)

- **What to do:** Determine if your project is doable. State whether this is a new system or an improvement on an existing one.
- **Output:** A document including a Sprint Roadmap showing key milestones and delivery dates.
- **Conducting the Project:** Use AI to help identify potential project risks (e.g., "What are the common technical hurdles for a real-time tracking app?").

1.3 Design Specification: Low-Level (20 Marks)

- **What to do:** This is the core of your "No Coding" project. You must design the system architecture, sequence diagrams, class diagrams.
- **Conducting the Project:** Use `Mermaid.js` to generate diagrams from text descriptions. Ensure your class diagram includes all relationships, operations/methods and attributes and the class diagram must connect to sequence diagram. Your system architecture explains based on C4-model diagram (Context-Container-Component-Code).
- **2026 Standard:** Focus on an API-First design—describe how your components talk to each other via REST or GraphQL.

1.4 Technical Documentation & Modern Stack (10 Marks)

- **What to do:** List your tools, programming languages, and environment. Include an E-R Diagram for your database.
- **2026 Standard:** Include a section on system architecture stack and a section on Containerization and their diagrams. Explain how you would use Docker to package your services.
- **AI Integration:** Use AI to help generate the initial Data Schema based on your class diagrams.

1.5 User Documentation & Prototyping (10 Marks)

- **What to do:** Create a user manual with screenshots of the Graphical User Interfaces (GUIs).
- **Conducting the Project:** Instead of static drawings, use Figma to create an interactive prototype. Take screenshots of these prototypes for your report.

1.6 Software Test Plan: "Shift-Left" Approach (10 Marks)

- **What to do:** Design tests for correctness, performance, UI, and robustness (handling incorrect data).
- **Conducting the Project:** Use AI to generate a list of "edge cases" or "negative test scenarios" that a human might forget.

1.7 DevOps: Integration & Deployment Plan (15 Marks)

- **What to do:** Detail how the code moves from a developer's machine to the cloud.
- **Output:** A diagram of your CI/CD Pipeline (e.g., GitHub Actions or GitLab CI).
- **2026 Standard:** Include a Deployment and Maintenance Plan that describes how you will deploy the software/system and monitor the system after it's "live".

1.8 Software Cost Estimation: FinOps Edition (10 Marks)

- **What to do:** Perform a software cost calculation. In 2026, this must include both developer labor and estimated cloud consumption costs (e.g., AWS or Azure monthly fees).

2. Important Dates & Submission

- **May 2, 12:00 PM: Project Title & Scope**
Submit via MS Teams. A 5-mark deduction applies to late submissions.
 - **July 9, 12:00 PM: Final Report & Presentation**
Submit `groupXX_project.docx` and `groupXX_slides.pptx`.
-

3. Presentation Strategy

- **Time:** 10 minutes maximum (including Q&A).
- **Role:** You are a Software Engineer pitching a ready-to-build design to your Customer.
- **Group Size:** Maximum 5 members.

Grading Rubric (Marking Scheme)

Section	Weight	Excellent (80–100%)	Good (60–79%)	Satisfactory (40–59%)	Poor (<40%)
1.1 Problem & AI Integration	5	Clearly defined problem; AI tools used effectively for brainstorming.	Good problem definition; minor mention of AI tools.	Vague problem statement; minimal AI usage.	Unclear objectives; no AI tool usage.
1.2 Agile Roadmap	10	Comprehensive Product Backlog with INVEST user stories; clear 3-sprint roadmap.	Backlog exists but lacks depth; basic roadmap.	Simple task list; no clear Agile methodology.	No roadmap or backlog provided.
1.3 Architecture & Design	20	Professional UML (Sequence, Class); modern API-first or Microservices design.	Correct UML; standard monolithic design.	Basic diagrams; inconsistencies between models.	Incomplete or incorrect diagrams.
1.4 Tech Stack & Cloud-Native	10	Modern stack defined; clear E-R diagram; Containerization (Docker) included.	Standard stack; correct E-R diagram.	Vague tech stack; missing database details.	Poorly defined tools and data schema.
1.5 Interactive Prototype	10	High-fidelity interactive prototype (Figma); excellent user flow.	Functional screenshots; clear manual.	Static screenshots; minimal user guidance.	Poor UI design; no user manual.
1.6 AI-Enhanced Testing	10	Robust test plan; AI used to generate complex edge cases; clear metrics.	Good test plan covering unit and integration tests.	Basic correctness testing only.	Minimal or irrelevant testing plan.
1.7 DevOps & CI/CD	15	Clear CI/CD pipeline diagram; Infrastructure as Code (IaC) concept; Maintenance plan.	Basic integration and deployment plan.	Generic installation instructions.	Missing deployment/integration details.
1.8 FinOps & Cost	10	Detailed calculation of labor + Cloud consumption (API/Compute) costs.	Basic labor cost calculation.	Rough estimate without breakdown.	No cost estimation provided.

Short Summary of Report

This is a comprehensive example of the MediDrone project report and presentation, designed as a reference for students in a Software Engineering course in 2026. This example follows the Agile and DevOps structure previously outlined.

Project Title: "MediDrone" – Rural Healthcare Delivery Coordinator

1.1 Problem Definition & AI-Assisted Ideation

Problem Statement: In rural regions of Thailand, "last-mile" delivery for critical medical supplies (e.g., anti-venom, cold-chain vaccines, emergency blood) often takes 4–6 hours due to difficult terrain.

- **Objectives:** To design an autonomous drone coordination system that reduces delivery time to under 30 minutes.
 - **AI Tooling:** We used Claude 3.5 to simulate rural logistics bottlenecks and ChatGPT-4o to refine our 15 core User Stories into a prioritized Product Backlog.
- **In-depth Objective:** The system must adhere to the "Golden Hour" of emergency medicine—ensuring that from the moment a request is logged, the drone is airborne within 120 seconds.
 - **Target Metrics:** 99.9% uptime for the Request Service; <30ms latency for AI pathfinding updates.
 - **AI Tooling Artifact:** Students should include a "Prompt Library" in the appendix, showcasing the specific system prompts used to generate the project's logic (e.g., "Act as a drone flight safety officer and identify three edge-case failures for a mountain delivery...").

1.2 Agile Roadmap & Project Plan

Scope: New system design. Students must provide backlogs

Product Backlog (High Priority):

- **User Story:**
 1. As a Doctor, I want to request emergency supplies via a mobile app.
 2. As a Dispatcher, I want to monitor drone flight health and battery status in real-time.
 3. As a System, I want to automatically re-route drones based on real-time weather data.
 4. As a Dispatcher, I want to see a 'Heat Map' of active requests so I can prioritize drone allocation during mass-casualty events.
- **Technical Debt Task:** Infrastructure setup for the Vector Database to store regional flight patterns.
- **Definition of Done (DoD):** A feature is "designed" only when its API contract is documented, its sequence diagram is drawn, and its test cases are defined.

Sprint Roadmap

- Sprint 1: Emergency Request Interface & Dispatcher Logic.
- Sprint 2: Navigation API Integration & Weather-Response Architecture.
- Sprint 3: Fleet Monitoring & Observability Dashboard Design.

1.3 Design Specification

Software Architecture: A Cloud-Native Microservices architecture.

- **Request Service:** Manages doctor orders.
- **Flight Agent (AI):** A Python-based service that interacts with LLMs to interpret complex flight regulations and weather alerts for pathfinding.
- **Telemetry Service:** Processes real-time GPS/Sensor data. UML Diagrams:
- **Sequence Diagram:** Models the interaction between the Mobile Client → API Gateway → Flight Agent → Drone Hardware Interface.
- **Class Diagram:** Focuses on entities like Drone, MedicalPackage, FlightPath, and EmergencyRequest.

Advance Software Architecture, we move from A Cloud-Native Microservices architecture to Agentic Workflows.

- **The AI Flight Agent:** This is not a simple script. It is an agent that monitors three data streams: (1) Live Weather API, (2) Civil Aviation Authority (CAA) REST API for "No-Fly Zones," and (3) On-board Telemetry.
- **Component Interaction:** Describe how the Vector Database (Pinecone) acts as "Long-term Memory," allowing the AI to retrieve how it successfully navigated a similar thunderstorm in that specific valley six months ago.

1.4 Technical Documentation & Tech Stack

- **Environment:** Node.js (TypeScript) for high-concurrency telemetry; Python for the AI flight agent.
- **Database:** PostgreSQL (with PostGIS) for geospatial historical data; Pinecone (Vector DB) for storing historical weather-pattern embeddings.
- **Containerization:** All services are designed to run in Docker containers, managed by Kubernetes.
- **Edge Computing:** Explain why part of the logic resides on the drone (C++/Rust for low-latency flight control) while the orchestration resides in the cloud (Python/Node.js).
- **Infrastructure as Code (IaC):** Students should provide a conceptual Terraform or Pulumi snippet for provisioning the "Request Service" in a serverless environment (e.g., AWS Lambda or Google Cloud Functions).

1.5 User Documentation & Prototyping

Interactive Prototype: A Figma prototype was created showing:

1. **Doctor Interface:** A simple "One-Tap Request" button with a real-time drone map tracker.
2. **Dispatcher Interface:** A high-level view of all active drones with color-coded battery alerts.

3. **AI-Enhanced UX:** We used AI heat-map generators to ensure the "Request Emergency" button is positioned for zero-latency user interaction.

1.6 Software Test Plan (AI-Enhanced)

Test Suites:

- **Correctness:** Automated Unit tests for the "Estimated Time of Arrival" (ETA) algorithm.
- **Robustness (AI-Generated):** We used AI to generate 50 failure scenarios, such as "Sudden 50km/h headwind" and "GPS signal jamming in mountain valleys," to test the system's fail-safe logic.
- **Performance:** Benchmarking the system to handle 1,000 concurrent drone telemetry streams.
- **Security (Red Teaming):** How does the design prevent "Signal Hijacking"? Include a plan for end-to-end encryption of the telemetry stream.
- **AI Evals:** A table showing "Expected AI Response" vs. "Potential Hallucination" and how the software validates the AI's pathing suggestions.

1.7 DevOps: Integration & Deployment Plan

Option 1: CI/CD Pipeline - Designed using GitHub Actions.

- **Build:** Every push to `main` triggers a Docker build.
- **Test:** Automated test suites run within the container.
- **Deploy:** Auto-deployment to an AWS EKS (Elastic Kubernetes Service) staging environment.
- **Monitoring:** Using Prometheus and Grafana to monitor drone response times and server health.

Option 2: CI/CD Pipeline Designed Stages using Opensource Tools:

- **Lint/Format:** Automated code quality checks.
- **Security Scan:** Automated vulnerability check (Snyk/Trivy).
- **Container Build:** Multi-stage Docker build to keep images small.
- **Blue-Green Deployment:** Explain how the new version of the Dispatcher app is rolled out without downtime.

1.8 FinOps & Cost Estimation

- **Human Capital:** Estimated 4,500 man-hours @ \$40/hr = \$180,000.
- **Cloud Infrastructure (2026 Rates):**
 - AWS Kubernetes Cluster: \$150/month.
 - Vector DB (Pinecone): \$70/month.
 - AI Token Usage (Inference): \$50/month.
- **Conclusion:** Total monthly operational cost estimated at \$270/month, making the system highly sustainable for provincial health departments.